

Xe buýt

Số lượt khách đã đi xe buýt chính là số khách lên xe buýt ở từng bến
 $= b_1 + b_2 + \dots + b_n$

Để tìm số lượng khách đông nhất tại một thời điểm, ta đơn giản duy trì một biến đếm số khách, giả sử là cnt . Ta duyệt qua từng bến và tính số khách có trên xe tại bến đó. Mỗi khi đến một bến i thì số khách trên xe lúc này sẽ là $cnt = cnt - a_i + b_i$

Ta sử dụng thêm 1 biến $maxx$ để lưu lại giá trị lớn nhất của cnt .

Cắt cây

Ở bài này, đề bài đã đảm bảo cho chúng ta rằng phần đất bị thu hồi là một đa giác (có các cạnh không tự cắt) $\Rightarrow$ Ta xét từng cạnh xem là cạnh này sẽ đi qua bao nhiêu điểm có tọa độ nguyên. Ở đây, ta không cần tìm ra các điểm này mà chỉ cần tìm số lượng mà thôi.

Ta xét 2 cạnh nối giữa 2 điểm (x_i, y_i) và (x_{i+1}, y_{i+1}) . Số lượng điểm có tọa độ nguyên nằm trên cạnh này là $\gcd(|x_i - x_{i+1}|, |y_i - y_{i+1}|)$ (Không tính 2 đầu mút).

Dự án

Dễ thấy, ta có 3 loại dự án là: dự án lãi ($a_i < b_i$), dự án hòa vốn ($a_i = b_i$) và dự án lỗ ($a_i > b_i$)

Trong 3 loại dự án này, rõ ràng ta sẽ đi tìm các dự án lãi để làm trước nhằm tăng số vốn S của chúng ta lên lớn nhất có thể do 2 loại dự án kia chỉ có làm số vốn của ta giữ nguyên hoặc giảm đi mà thôi.

Với các dự án lại ta ưu tiên làm các dự án có a_i nhỏ hơn trước, điều này vẫn đảm bảo số lượng dự án tối đa của chúng ta do:

Giả sử có 2 dự án lãi i và j có $a_i < a_j$:

Nếu $S_{cũ} > a_j (> a_i)$ thì ta làm a_i trước thì $S_{mới} = S_{cũ} - a_i + b_i > S_{cũ} > a_j$ nên hoàn toàn có thể làm được a_j

Nếu $S_{\text{cũ}} < a_j$ thì rõ ràng ta phải đi làm i trước và mong đợi rằng $S_{\text{mới}}$ có thể lớn hơn a_j

=> Ở bước này, ta duyệt qua từng dự án có lãi (tăng dần theo a_i) và đếm số lượng dự án làm được

Sau đó, ta với số vốn S lúc này, ta đi thực hiện các dự án hòa vốn, tất nhiên ta chỉ có thể thực hiện được các dự án có $a \leq S$. Sau khi thực hiện các dự án này thì số vốn vẫn là S .

Cuối cùng, ta thực hiện các dự án lỗ. Với các dự án này, ta có thể bỏ qua luôn những dự án có $a > S$ vì ta không thể nào thực hiện được những dự án này (do S sẽ chỉ có giảm mà thôi). Với những dự án lỗ mà $a \leq S$, ta sẽ ưu tiên thực hiện các dự án có b_i lớn hơn trước, vì:

Giả sử có 2 dự án lỗ i và j có $b_i > b_j$ ($S \geq a_i, a_j$):

Xét ta thực hiện i thì: $S_{\text{mới}} = S_{\text{cũ}} - a_i + b_i$

Nếu $S_{\text{mới}} \geq a_j$ thì có thể làm j được

Nếu $S_{\text{mới}} < a_j$ thì ta không làm j được, bây giờ ta chứng minh là trong trường hợp này nếu làm ngược lại (j trước i thì cũng không thể làm được cả 2):

Muốn làm được cả 2 thì:

$$S_{\text{cũ}} - a_j + b_j \geq a_i$$

$$S_{\text{cũ}} - a_i + b_j \geq a_j$$

Mà $b_i > b_j$: => $S_{\text{cũ}} - a_i + b_i > S_{\text{cũ}} - a_i + b_j \geq a_j$, tức: $S_{\text{mới}} > a_j$ (mâu thuẫn với giả sử)

Sort giảm dần theo b .

Lưu lại danh sách những dự án lỗ mà mình đã làm.

Xét đến dự án i : $\text{loss} = a_i - b_i$

Nếu vẫn làm được dự án i : cho vào danh sách

Nếu không làm được dự án i : Có khả năng do đã làm 1 dự án bị lỗ quá lớn trước đó => Thay dự án lỗ nhất ở trong danh sách = i .

Thay được khi: $S + \text{loss}' \geq a_i$

Sử dụng priority_queue để duy trì danh sách

Sắp xếp xâu

Đầu tiên, ta nhận thấy điều kiện để có thể sắp xếp các ký tự này lại sao cho không có 2 ký tự liên tiếp giống nhau là không có ký tự nào có số lượng lớn hơn một nửa tổng số ký tự:

Gọi $\text{cnt}[c]$ là số lượng ký tự c trong xâu thì: $\max(\text{cnt}[c]) \leq (|S| + 1) / 2$ (làm tròn lên)

Với điều kiện tạo ra xâu T có thứ tự từ điển nhỏ nhất, ta sẽ duyệt lần lượt từng vị trí của T : xét đến vị trí thứ i của T , ta sẽ duyệt để kiểm tra xem có thể điền vào vị trí i này của T ký tự c hay không.

For $i: 1 \rightarrow n$

For $c: a \rightarrow z$

/// Ta cần kiểm tra xem có điền c vào vị trí i được không, nếu được thì dùng luôn do c ta đang for từ nhỏ đến lớn

Điều kiện để có thể điền c vào vị trí i là:

- c khác ký tự liên trước T_{i-1} : $c \neq T_{i-1}$
- Vẫn còn ký tự c để dùng: $\text{cnt}[c] > 0$
- Tồn tại cách sắp xếp cho các ký tự ở sau:

$\max(\text{cnt}) \leq (|S| - i + 1) / 2$: điều kiện để ở sau có thể sắp xếp không để 2 ký tự liên tiếp giống nhau, ở đây $\text{cnt}[c]$ đã giảm đi 1 (dùng ở vị trí i rồi). Tuy nhiên điều kiện kiểm tra này chưa đủ do ta còn cần kiểm tra xem có cách nào mà ký tự $T_{i+1} \neq c$ hay không nữa.

Các ký tự c còn lại được điền vào các vị trí từ $i + 2$ đến n (và không có 2 ký tự c nào cạnh nhau) thì chỉ có tối đa là $(|S| - i) / 2$ vị trí mà thôi $\Rightarrow \text{cnt}[c] \leq (|S| - i) / 2$. Còn các ký tự còn lại thì đặt thoải mái nên $\text{cnt} \leq (|S| - i + 1) / 2$

Rải sỏi

Giả sử ta chọn đoạn $[l, r]$ và chọn giá trị k để thực hiện thao tác ($k < 0$ coi như là rút bớt k viên sỏi mỗi ô, $k > 0$ coi như là thêm k viên sỏi mỗi ô), khi này số ô có x viên sỏi sẽ được tính = số ô có x viên sỏi nằm ngoài khoảng $[l, r]$ + số ô có $x - k$ viên sỏi nằm trong khoảng $[l, r]$

Để tính phần thứ nhất, ta hoàn toàn có thể chuẩn bị được mảng $cnt[i]$ là số lượng ô có x viên sỏi từ ô 1 đến ô i

=> Số ô có x viên sỏi nằm ngoài khoảng = $cnt[l-1] + cnt[n] - cnt[r] = cnt[n] - (cnt[r] - cnt[l-1])$ chính là lấy tổng số ô có x viên sỏi trừ đi số ô có x viên sỏi nằm trong khoảng.

Xét đến phần còn lại, ta có nhận xét, với giá trị k , gọi $x' = x - k$ thì ta chỉ cần quan tâm đến những đoạn có $a_l = a_r = x'$ mà thôi. Điều này có nghĩa các giá trị khác x' không có ảnh hưởng gì đến đáp án, nên ta không cần quan tâm

=> Ta chuẩn bị mảng / vector pos chứa vị trí xuất hiện của x' trong mảng a .

Như đã nói, ta duyệt qua các đoạn có vị trí 2 đầu mút = x'

/// k là số phần tử của pos (chính là số lần xuất hiện của x')

For $L: 1 \rightarrow k$

For $R: L \rightarrow k$

$i = pos[L]$

$j = pos[R]$

/// Có $R - L + 1$ ô có x' viên sỏi trong đoạn đang xét này

$ans = cnt[n] - (cnt[j] - cnt[i - 1]) + R - L + 1$

Ta có thể tính nhanh được giá trị ans bằng cách như sau

For $R: 1 \rightarrow k$

$maxx = \max(maxx, (cnt[pos[R] - 1] - (R - 1)))$

$ans = cnt[n] - (cnt[pos[R]] - R) + maxx$

Do ta duyệt R tăng dần nên có thể kết hợp trong quá trình duyệt trước đó, tính giá trị maxx là giá trị lớn nhất khi chọn 1 vị trí ở phía trước làm L